

Low-Rank Adaptation and Sparse Optimization

Abhinav Saraf

Abstract

Parameter-efficient fine-tuning (PEFT) methods enable adaptation of large pretrained models by updating only a small subset of parameters. In this study, we compare LoRA, AdaLoRA, and Sparse Low-Rank Adaptation (SoRA) on the CoLA task using DeBERTa-v3-base, evaluating trade-offs between performance, parameter efficiency, and effective rank. We further investigate the optimization dynamics of sparse gating in SoRA by comparing proximal gradient updates against standard SGD with ℓ_1 subgradients. We also explore the application of these methods on an xLSTM.

1 Introduction

Current large language models have demonstrated a strong performance across a wide range of natural language understanding tasks. However, adapting these models to downstream tasks through full fine-tuning is very computationally expensive, requiring large memory and storage resources. This has motivated the development of parameter-efficient fine-tuning (PEFT) methods, which aim to achieve similar performance while updating only a small subset of model parameters.

Among these approaches, Low-Rank Adaptation (LoRA) (Hu et al., 2022) is the widely adopted method that injects trainable low-rank matrices into pre-trained weight updates, reducing the number of trainable parameters by a huge margin. Building on this idea, AdaLoRA (Zhang et al., 2023) introduces adaptive rank allocation, dynamically redistributing the parameter budget across layers based on their importance during training. More recently, Sparse Low-Rank Adaptation (SoRA) (Ding et al., 2023) combines low-rank parameterization with sparsity, further reducing the effective number of parameters while preserving performance.

These methods are also different in their optimization dynamics. In particular, SoRA introduces sparsity-inducing regularization, making the choice of optimization algorithm more critical. While standard stochastic gradient descent (SGD) or Adam-based optimizers are commonly used, proximal gradient methods offer a principled approach for handling sparsity constraints.

In this work, we conduct a comparative study of LoRA, AdaLoRA, and SoRA on the CoLA (Warstadt et al., 2019) benchmark, evaluating their performance in terms of accuracy (measured via Matthews Correlation Coefficient), parameter efficiency, and training cost. Further, we compare standard gradient-based optimization with proximal gradient descent, highlighting differences in convergence behavior, sparsity patterns, and overall efficiency.

2 Background

2.1 Low-Rank Adaptation (LoRA)

LoRA is a parameter-efficient fine-tuning method motivated by the empirical observation that weight updates obtained during full fine-tuning are often intrinsically low-rank (Aghajanyan et al., 2021, Li et al., 2018). Instead of updating the full weight matrix $W \in \mathbb{R}^{d \times k}$, LoRA models the update ΔW using a low-rank decomposition (Hu et al., 2022):

$$\Delta W = BA, \tag{1}$$

where $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$, and $r \ll \min(d, k)$. The adapted weight matrix is then:

$$W' = W + \Delta W = W + BA. \quad (2)$$

In practice, a scaling factor α is used to control the update magnitude:

$$W' = W + \frac{\alpha}{r}BA. \quad (3)$$

During training, the original weights W are frozen, and only the low-rank matrices A and B are optimized, significantly reducing the number of trainable parameters. Given an input x , the layer output becomes:

$$y = Wx + \frac{\alpha}{r}BAx. \quad (4)$$

This formulation also enables modular task adaptation. The base model W remains unchanged, and task-specific behavior is captured entirely by the low-rank updates. As a result, different tasks can be handled by swapping the corresponding (A, B) matrices without storing multiple full model copies.

2.2 Adaptive Budget Allocation (AdaLoRA)

While LoRA uses a fixed rank r across all layers, this uniform allocation may not be optimal, as different layers contribute unequally to a task. AdaLoRA addresses this limitation by dynamically allocating the rank budget across layers based on their relative importance during training (Zhang et al., 2023).

Similar to LoRA, the weight update for a layer is parameterized as:

$$\Delta W = BA, \quad (5)$$

where $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$. However, instead of fixing r , AdaLoRA introduces a global rank budget R and redistributes it across layers by adjusting the effective rank r_ℓ for each layer ℓ such that:

$$\sum_{\ell} r_{\ell} \leq R. \quad (6)$$

To determine layer importance, AdaLoRA maintains a singular value decomposition (SVD) of the low-rank updates and evaluates the contribution of each singular component. As $\Delta W = U\Sigma V^\top$, the singular values in Σ are used to guide rank allocation by removing components with small magnitude.

The adapted weight matrix becomes:

$$W' = W + \frac{\alpha}{r_{\ell}}B_{\ell}A_{\ell}, \quad (7)$$

where r_{ℓ} varies across layers. This adaptive strategy enables better utilization of the parameter budget by concentrating capacity where it is most needed.

2.3 Sparse Low-Rank Adaptation (SoRA)

Sparse Low-Rank Adaptation (SoRA) extends low-rank adaptation by incorporating structured sparsity through a learnable gating mechanism (Ding et al., 2023). Similar to LoRA, the weight update is parameterized as:

$$\Delta W = BA, \quad (8)$$

where $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$. To induce sparsity, SoRA introduces a gating vector $g \in \mathbb{R}^r$ that regulates the contribution of each rank component:

$$\Delta W = B \text{diag}(g) A. \quad (9)$$

Here, each element g_i controls whether the corresponding rank-1 component is active. To encourage sparsity in the gates, the training objective is augmented with an ℓ_1 regularization term:

$$\min_{A, B, g} \mathcal{L}(W + B \text{diag}(g) A) + \lambda \|g\|_1, \quad (10)$$

where $\mathcal{L}$ is the task loss and λ controls the sparsity level. The adapted weight becomes:

$$W' = W + \frac{\alpha}{r} B \text{diag}(g) A. \quad (11)$$

Due to the non-smooth ℓ_1 penalty, proximal gradient methods (Parikh and Boyd, 2014) are well-suited for optimizing the gating variables. The update for g takes the form:

$$g^{(t+1)} = \text{prox}_{\eta\lambda} \left(g^{(t)} - \eta \nabla_g \mathcal{L} \right), \quad (12)$$

where the proximal operator corresponds to element-wise soft-thresholding:

$$\text{prox}_\tau(x) = \text{sign}(x) \cdot \max(|x| - \tau, 0). \quad (13)$$

By combining low-rank decomposition with learnable gating, SoRA can selectively deactivate redundant rank components, achieving a more compact and efficient adaptation compared to fixed-rank methods.

3 Experimental Setup

All experiments are conducted on the CoLA task (Warstadt et al., 2019) which is a binary sequence classification task in which the model has to identify if the given sentence is grammatically correct. We use DeBERTaV3 (He et al., 2020) as our base model (184M) and its corresponding tokenizer which has a vocab size of 128K.

Method	MCC	Trainable Params	Effective Rank	Training Time (mins)
Full Fine-tuning	0.693	100%	-	11.1
LoRA	0.623	0.24%	8	18.3
AdaLoRA	0.598	0.47%	8	25.97
SoRA	0.650	0.71%	2.74	32

Table 1: Comparison of PEFT methods on CoLA.

Among the evaluated PEFT methods, SoRA achieved the highest MCC score (0.650) while reducing the effective adaptation rank from 8 to 2.74. This indicates that the adaptation required for CoLA is actually much smaller than our chosen rank budget. In contrast, AdaLoRA retained the full rank budget and did not achieve comparable performance improvements.

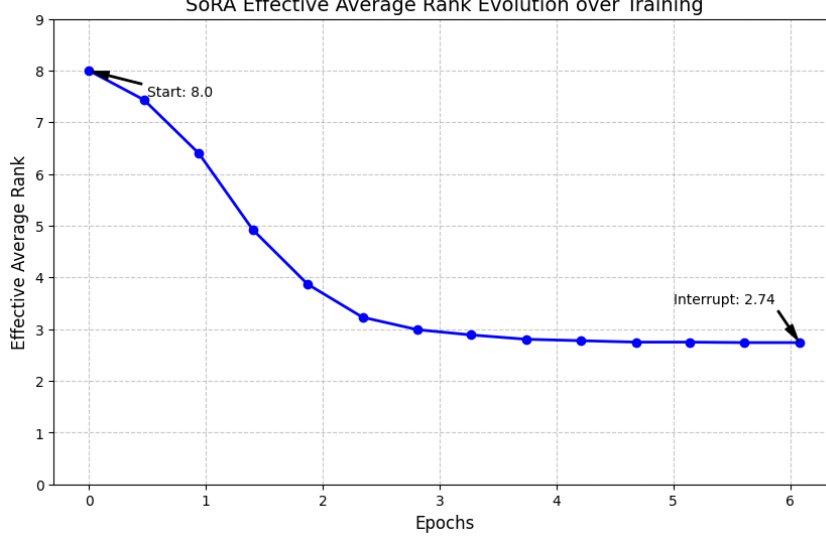


Figure 1: Trajectory of the effective average rank across training epochs using Sparse Low-rank Adaptation (SoRA). The model initializes with a maximum rank configuration of 8. As training progresses, the sparsity penalty actively forces the gating values toward zero, rapidly reducing parameters during the first three epochs before converging to a rank of 2.74.

4 Sparse Optimization Analysis

4.1 Standard SGD with ℓ_1 Subgradient

The binary cross-entropy loss L_{BCE} is computed as an average over a batch of N samples, where each sample is indexed by $i = 1, \dots, N$. For each sample, $y_i \in \{0, 1\}$ denotes the ground truth label, while z_i represents the raw logit (unnormalized model output). This logit is passed through the sigmoid function $\sigma(z_i)$ to produce a predicted probability $p_i = \sigma(z_i)$, which estimates the likelihood that the sample belongs to the positive class.

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)], \quad p_i = \sigma(z_i) \quad (14)$$

Using the Chain Rule, we can split the derivative into two parts:

$$\frac{\partial L_i}{\partial z_i} = \frac{\partial L_i}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_i} \quad (15)$$

Computing the derivative of the loss with respect to the probability (p_i):

$$\begin{aligned} \frac{\partial L_i}{\partial p_i} &= -\left(\frac{y_i}{p_i} - \frac{1 - y_i}{1 - p_i}\right) \\ \Rightarrow \frac{\partial L_i}{\partial p_i} &= \frac{-y_i(1 - p_i) + p_i(1 - y_i)}{p_i(1 - p_i)} \\ \Rightarrow \frac{\partial L_i}{\partial p_i} &= \frac{-y_i + y_i p_i + p_i - y_i p_i}{p_i(1 - p_i)} \\ &= \frac{p_i - y_i}{p_i(1 - p_i)} \end{aligned}$$

Computing the derivative of the sigmoid function with respect to the logit (z_i):

$$\begin{aligned}
p_i &= (1 + e^{-z_i})^{-1} \\
\Rightarrow \frac{\partial p_i}{\partial z_i} &= -(1 + e^{-z_i})^{-2} \cdot (-e^{-z_i}) \\
&= \frac{e^{-z_i}}{(1 + e^{-z_i})^2} \\
&= \frac{1}{1 + e^{-z_i}} \cdot \frac{e^{-z_i}}{1 + e^{-z_i}} \\
&= p_i(1 - p_i)
\end{aligned}$$

Substituting these results back into equation (15):

$$\begin{aligned}
\frac{\partial L_i}{\partial z_i} &= \left(\frac{p_i - y_i}{p_i(1 - p_i)} \right) \cdot p_i(1 - p_i) \\
\Rightarrow \frac{\partial L_i}{\partial z_i} &= p_i - y_i
\end{aligned}$$

The SGD update is of the form

$$g^{(t+1)} = g^{(t)} - \tau \nabla_g f(g).$$

The loss function f including the ℓ_1 regularization is:

$$\begin{aligned}
f(g) &= L_{\text{BCE}} + \lambda \|g\|_1 \\
&= -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)] + \lambda \|g\|_1
\end{aligned}$$

Choosing the subgradient at $g = 0$ to be 0, we obtain:

$$\nabla_g f(g) = \frac{1}{N} \sum_{i=1}^N (p_i - y_i) \frac{\partial z_i}{\partial g} + \lambda \text{sign}(g)$$

SGD form:

$$g^{(t+1)} = g^{(t)} - \tau \left[\frac{1}{N} \sum_{i=1}^N (p_i - y_i) \frac{\partial z_i}{\partial g} + \lambda \text{sign}(g^{(t)}) \right]$$

4.2 Proximal Gradient Descent

In this method, we combine a gradient descent step on the smooth loss with a proximal (shrink-age) step to handle the non-smooth ℓ_1 regularization.

The gradient descent step is:

$$s^{(t)} = g^{(t)} - \tau \left[\frac{1}{N} \sum_{i=1}^N (p_i - y_i) \frac{\partial z_i}{\partial g} \right]$$

For any function $h(g)$, the proximal operator (Parikh and Boyd, 2014) is defined as:

$$\text{prox}_{\tau h}(v) = \arg \min_g \left\{ \frac{1}{2} \|g - v\|_2^2 + \tau h(g) \right\}$$

Applying this to the ℓ_1 penalty:

$$\text{prox}_{\tau\lambda\|\cdot\|_1}(s) = \arg \min_g \left\{ \frac{1}{2}\|g - s\|_2^2 + \tau\lambda\|g\|_1 \right\}$$

Since the objective is separable, we solve for each component g_j :

$$\min_{g_j} \left\{ \frac{1}{2}(g_j - s_j)^2 + \tau\lambda|g_j| \right\}$$

Using the subgradient of the absolute value function,

$$\partial|g_j| = \begin{cases} \{1\}, & g_j > 0 \\ \{-1\}, & g_j < 0 \\ [-1, 1], & g_j = 0 \end{cases}$$

To find the minimum, we differentiate the equation, giving us 3 cases:

Case 1: $g_j > 0$

$$g_j - s_j + \tau\lambda = 0 \Rightarrow g_j = s_j - \tau\lambda, \quad s_j > \tau\lambda$$

Case 2: $g_j < 0$

$$g_j - s_j - \tau\lambda = 0 \Rightarrow g_j = s_j + \tau\lambda, \quad s_j < -\tau\lambda$$

Case 3: $g_j = 0$

Using $\partial|g_j| \in [-1, 1]$:

$$0 \in -s_j + \tau\lambda[-1, 1] \Rightarrow |s_j| \leq \tau\lambda$$

Combining all cases gives the soft-thresholding operator:

$$g_j = \begin{cases} s_j - \tau\lambda, & s_j > \tau\lambda \\ 0, & |s_j| \leq \tau\lambda \\ s_j + \tau\lambda, & s_j < -\tau\lambda \end{cases}$$

In compact form:

$$g^{(t+1)} = \text{sign}(s^{(t)}) \odot \max(|s^{(t)}| - \tau\lambda, 0)$$

4.3 Choice of subgradient and method comparison

Proximal gradient descent introduces sparsity by producing exact zeros when $|s_j| \leq \tau\lambda$, whereas SGD does not. The SGD update at $g = 0$ depends on which subgradient we pick in $[-1, 1]$. While different choices may lead to different trajectories, they do not alter the fundamental distinction between the methods and the sparsity inducing behaviour of proximal gradient descent is independent of the choice of subgradient.

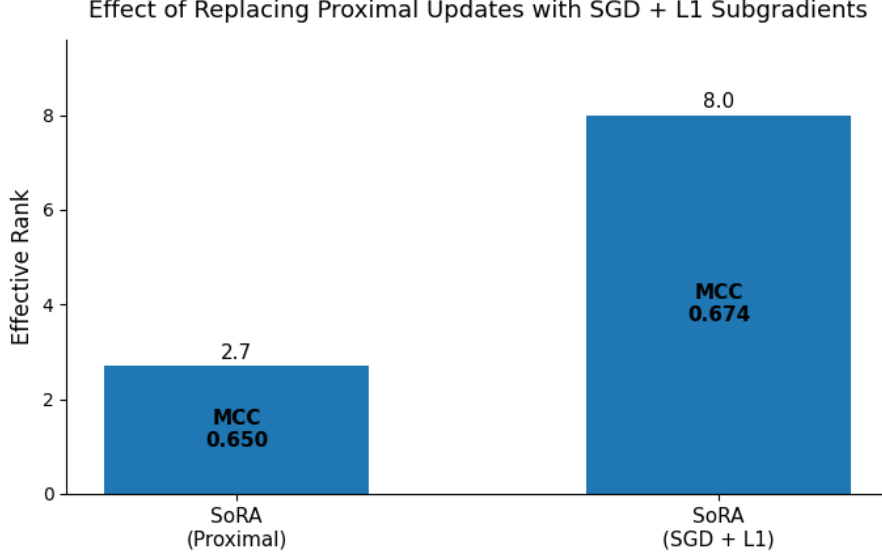


Figure 2: Comparison between proximal soft-thresholding and SGD with ℓ_1 subgradients in SoRA on the CoLA benchmark. Replacing the proximal update with an ℓ_1 subgradient update increases the effective rank from 2.7 to 8.0 while slightly improving MCC from 0.650 to 0.674. There is a tradeoff between sparsity and performance, and the proximal update prioritizes sparsity at the cost of performance.

5 Low rank adaptation on xLSTM

5.1 Background

The first building block of xLSTM is the scalar LSTM (sLSTM). Unlike conventional LSTMs that use sigmoid gates bounded between 0 and 1, sLSTM uses exponential gating mechanisms with normalization. This increases the dynamic range of memory updates and improves long-range information retention.

The memory update retains the LSTM form:

$$c_t = f_t \odot c_{t-1} + i_t \odot z_t, \quad (16)$$

where i_t and f_t are computed using exponentially parameterized gates. This modification allows the model to perform more flexible memory allocation while maintaining stable training.

The primary innovation of xLSTM is the matrix LSTM (mLSTM). Traditional LSTMs store memory as a vector-valued state. In contrast, mLSTM maintains a matrix-valued memory:

$$C_t \in \mathbb{R}^{d \times d}. \quad (17)$$

At each timestep, key and value vectors are generated:

$$k_t = W_k x_t, \quad (18)$$

$$v_t = W_v x_t. \quad (19)$$

These vectors are written into memory through an outer-product update:

$$C_t = f_t C_{t-1} + i_t (v_t k_t^\top). \quad (20)$$

Information retrieval is then performed using a query vector q_t :

$$r_t = C_t q_t. \quad (21)$$

The resulting mechanism is like attention-based memory retrieval while preserving the recurrent nature of the model.

5.2 Applying PEFT Methods to xLSTM

In Transformer architectures, PEFT methods such as LoRA are typically applied to the query, key, and value projection matrices. A similar approach can be adopted for xLSTM by targeting the large projection matrices responsible for memory formation and retrieval.

Within the sLSTM component, the input, forget, output, and candidate-state projections are given by

$$i_t = f(W_i x_t + U_i h_{t-1}), \quad (22)$$

$$f_t = f(W_f x_t + U_f h_{t-1}), \quad (23)$$

$$o_t = f(W_o x_t + U_o h_{t-1}), \quad (24)$$

$$z_t = f(W_z x_t + U_z h_{t-1}), \quad (25)$$

where W_* and U_* denote learnable projection matrices. Similarly, the mLSTM component contains key, value, and query projections

$$k_t = W_k x_t, \quad (26)$$

$$v_t = W_v x_t, \quad (27)$$

$$q_t = W_q x_t. \quad (28)$$

These matrices are the majority of the trainable parameters and directly influence how information is written to and retrieved from memory. Hence, they are natural targets for low-rank adaptation. Applying LoRA to xLSTM therefore involves replacing selected projection matrices such as W_k , W_v , W_i , and W_f with their low-rank infused versions.

For xLSTM, SoRA is also quite attractive because different memory operations may require different adaptation capacities. By sparsifying the low-rank factors associated with the gating and memory projection matrices, SoRA can identify the most important adaptation directions while maintaining less parameters.

6 Limitations

One limitation of this work is that all experiments were conducted solely on the CoLA task from the GLUE benchmark. Since CoLA focuses on linguistic acceptability, the observed trends may not necessarily generalize to other NLP tasks such as question answering, natural language inference, or text generation. Furthermore, due to computational constraints, only a limited

set of hyperparameters such as learning rate, sparsity coefficient λ , and adaptation rank were explored.

While we proposed a method for extending SoRA to xLSTM architectures, a complete experimental evaluation was not conducted. Hence, this work focuses primarily on the design and theoretical justification of the proposed approach.

7 Conclusion

Our experiments show that low-rank adaptation can achieve competitive performance while training only a small fraction of the model parameters. We also compared proximal soft-thresholding with standard SGD using ℓ_1 subgradients, where proximal updates produced significantly lower effective ranks by explicitly driving small gate values to zero. In contrast, SGD with ℓ_1 subgradients typically resulted in denser solutions and higher effective ranks.

Finally, we propose the extension of the low-rank adaptation framework to xLSTM architectures by applying low-rank sparse adapters to memory-related projection matrices. This shows that PEFT techniques are not restricted to Transformers and can be generalized to modern recurrent architectures.

Overall, our results show the effectiveness of PEFT methods as an alternative to full fine-tuning for reducing trainable parameters while still maintaining strong downstream performance.

References

- Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021*, pages 7319–7328. Association for Computational Linguistics, 2021. doi: 10.18653/V1/2021.ACL-LONG.568. URL <https://doi.org/10.18653/v1/2021.acl-long.568>.
- Ning Ding, Xingtai Lv, Qiaosen Wang, Yulin Chen, Bowen Zhou, Zhiyuan Liu, and Maosong Sun. Sparse low-rank adaptation of pre-trained language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 4133–4145, 2023.
- Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. DeBERTa: Decoding-enhanced BERT with disentangled attention. *CoRR*, abs/2006.03654, 2020. URL <https://arxiv.org/abs/2006.03654>.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2): 3, 2022.
- Chunyuan Li, Heerad Farkhoor, Rosanne Liu, and Jason Yosinski. Measuring the intrinsic dimension of objective landscapes. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018. URL <https://openreview.net/forum?id=ryup8-WCW>.
- Neal Parikh and Stephen Boyd. Proximal algorithms. *Foundations and Trends in optimization*, 1(3):127–239, 2014.
- Alex Warstadt, Amanpreet Singh, and Samuel R. Bowman. Neural network acceptability judgments. *Trans. Assoc. Comput. Linguistics*, 7:625–641, 2019. doi: 10.1162/TACL_A_00290. URL https://doi.org/10.1162/tacl_a_00290.
- Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.10512*, 2023.